Types of variable

1) Local variable
2) Global variable
3) Static variable

1) Local variable –

```
fun(int x)
{
  int y;

}
```

(a) Scope

→ with in a function or a block in which it is define.

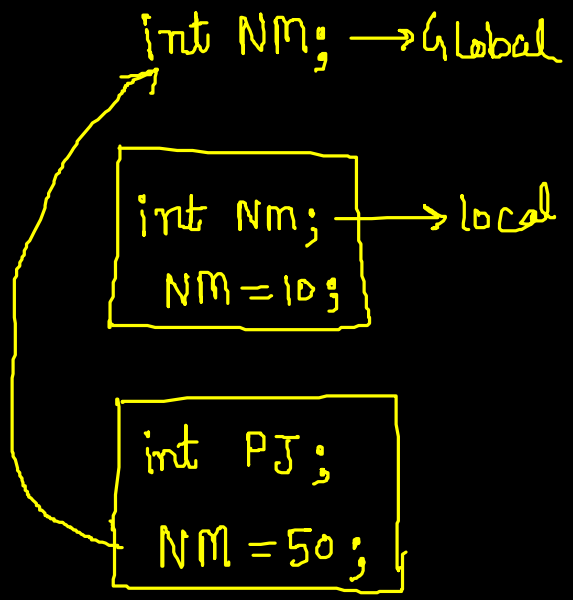
(b) Default value

Garbage.

```
fun(int x)
{ int y;
  {
    int z;
  }
}
```

Global variable :-

```
#include <stdio.h>
int x;          ——→ Global
int main()
{
  int x;        ——→ Local
}
int add()
{
  int a;        ——→ Local
}
```



a) Scope :-

Throught the program

b) Default :-

value = 0

3) Static Variable :-

```
main()
{
   fun();
   fun();
   fun();
}
```

```
Void fun()
{
   int static x = 5;
   printf("%d", x);
   x++;
}
```

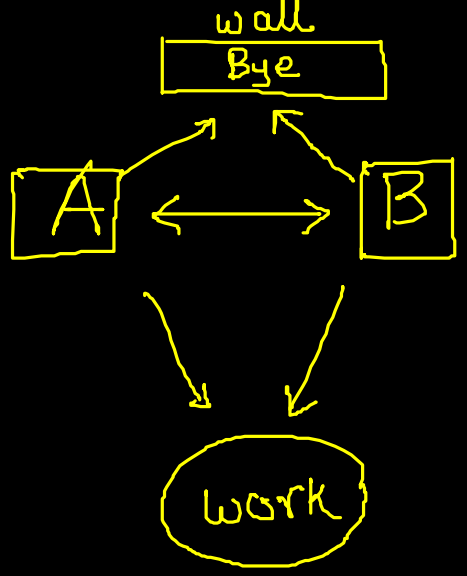
5, 6, 7

a) Scope :-

Throught the execution of program

b) Default :-

Zero

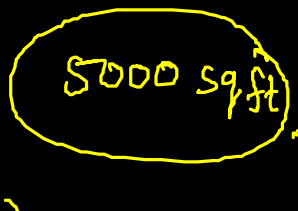


1) Msg passing

2) Shared variable.

Static Global

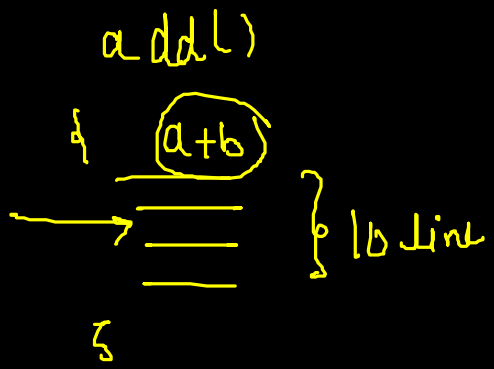
Advantages of Function

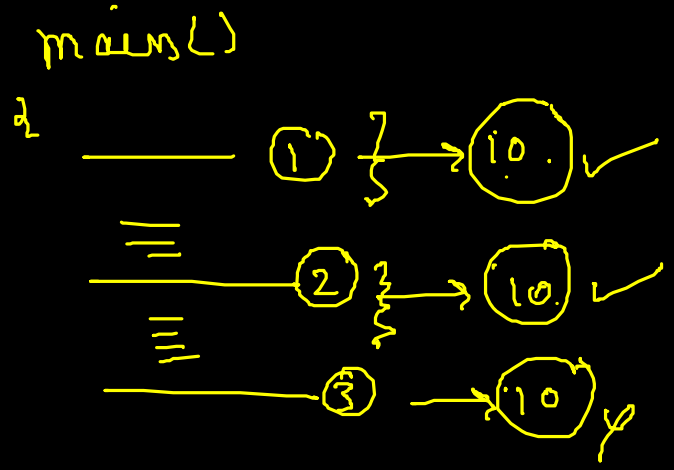


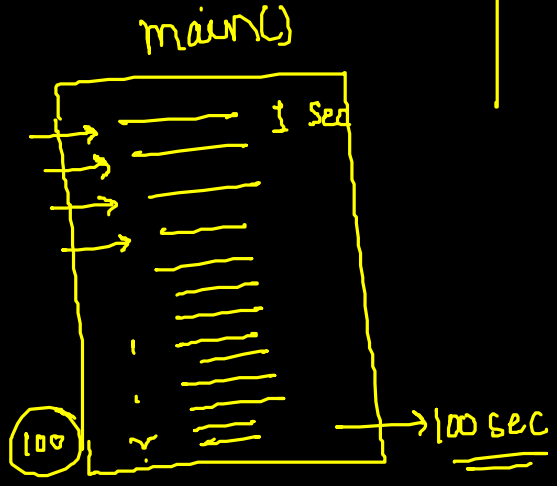
1) Code reusability
2) Space management
3) Readability
4) Easy to modify
5) Easy to Debug

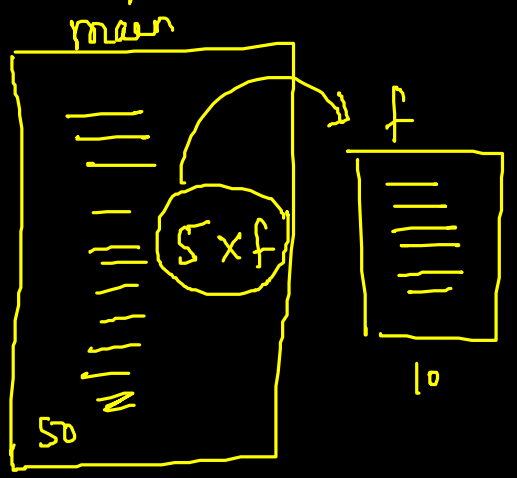
Disadvantages:-











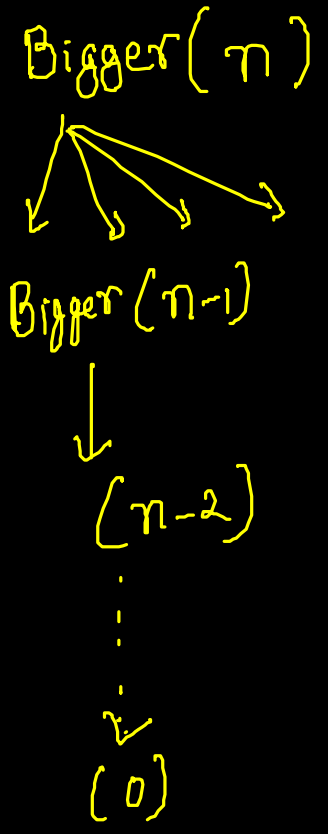
Recursion :- Function calling itself is called Recursion

```
fun( )
{
   fun( );
}
```

```
for(  ;  ;  )
{
   ===
   ===
   ===
}
```

condition

Repeated work

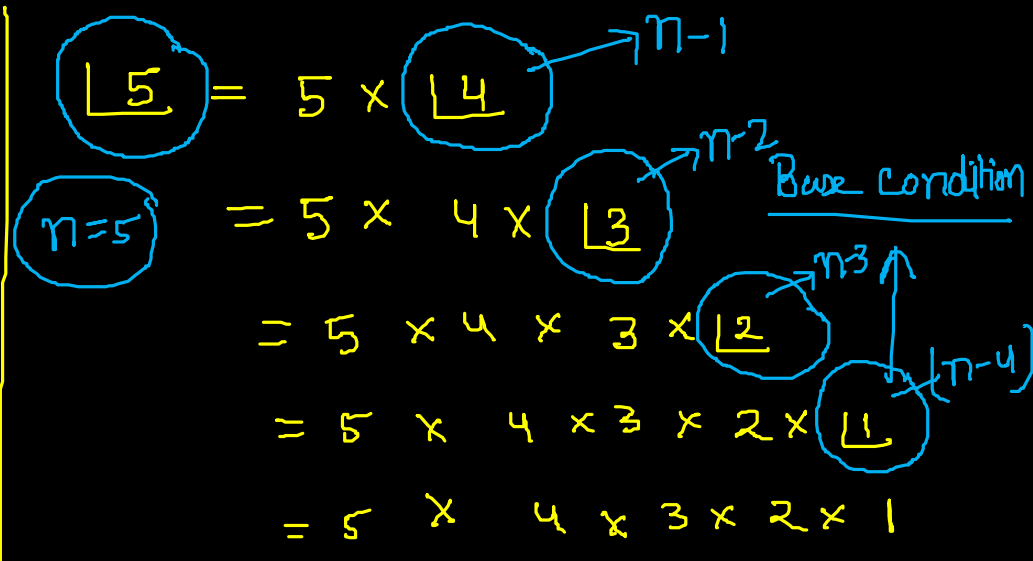


$\lfloor 5 = 5 \times 4 \times 3 \times 2 \times 1$

```
int main()
{
    int n=5, fact=1, i;

    for(i=1; i<=n; i++)
    {
        fact = fact *i;
    }
    printf(" factorial is %d ", fact);

    return 0;
}
```



$\lfloor 5 = 5 \times \lfloor 4$

$= 5 \times 4 \times \lfloor 3$

$= 5 \times 4 \times 3 \times \lfloor 2$

$= 5 \times 4 \times 3 \times 2 \times \lfloor 1$

$= 5 \times 4 \times 3 \times 2 \times 1$

$fact(5) = 5 \times fact(4)$

1) Base Condition → if(n == 1)
return 1;

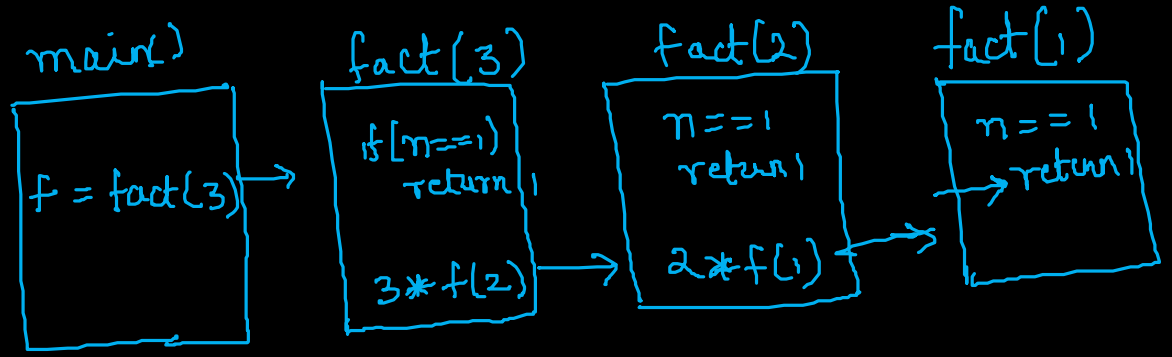
2) Recursion Condition → f(n) = n * f(n-1);

```
fact( int n)
{

return n * fact(n-1);

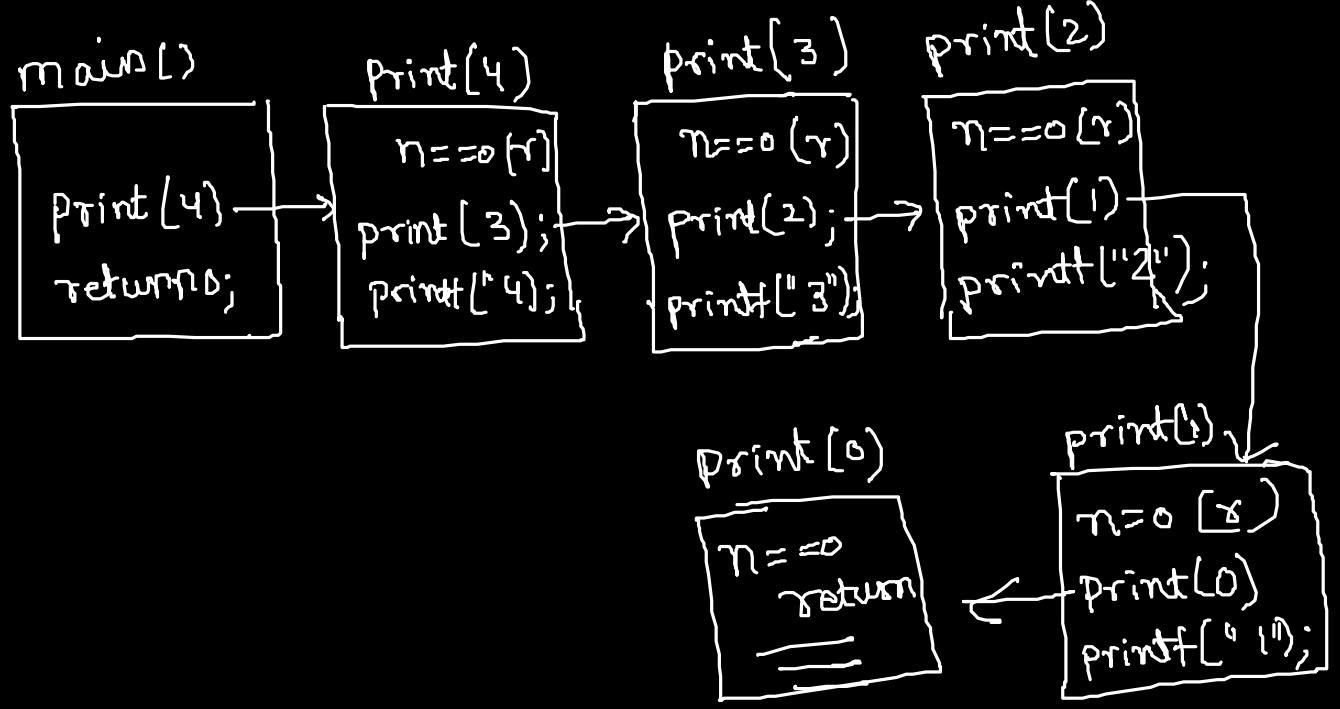
}
```

```
fact(n)
{
  if (n==1)
    return 1;
  return  n*fact(n-1);
}
int main()
{
   int f = fact(3);
}
```



WAP to print first n natural no. using recursion.

```
int main()
{
  print(n);
  return 0;
}
void print(int n)
{  if(n==0) return ;
   print(n-1);
   printf("%d", n);
}
```



1, 1, 2, 3, 5, 8,

```
fib(n)
{  if(n==1 || n==0)
       return 1;
printf(fib(n-1) + f(n-2))

}
```

Array :-

`x[1] = 5;`

`int x, y, z;`

`a[5] = 10;`

`int [x1, x2, x3, x4, ... ..., x1000];`



Syntax :-

`Data_Type variable_name[size];`

`int x[10];` → `x[0], x[1], x[2], x[3], ..... , x[9]`

4 byte x 10 = 40 bytes